

CH09 模块化与组件化开发

— 期末复习要点总结

计算机学院 | 软件开发架构平台 | 综合整理自教材 pp.2-40 (共 40 页幻灯片)

内容大纲:

- 一、前端模块化概述 [p.2]
- 二、同步与异步编程基础[pp.7-9]
- 三、Promise 对象详解[pp.6-15]
- 四、async / await 关键字[pp.18-21]
- 五、Fetch API[pp.23-25]
- 六、Axios 库[pp.28-29]
- 七、Fetch API 与 Axios 对比[pp.31-32]
- 八、项目案例与前端项目演进[pp.4, 34-39]
- 九、本章小结 [p.40]
- 附录 A: 核心 API 速查表
- 附录 B: Fetch vs Axios 对比表
- 附录 C: Promise 状态转换图

一、前端模块化概述 [p.2]

模块化的分类 [p.2]

- **外部模块化** (External Modularization): 指引入前端工程项目的一个或多个第三方包或插件, 一般由一个或多个 JavaScript 文件组成。 [p.2]
- **内部模块化** (Internal Modularization): 指前端工程项目内部的分层或分类, 一般内部一个模块由一个 JavaScript 文件表示。 [p.2]

模块化的主要内容 [p.2]

- **外部模块的管理**: 通过 Node.js 和 NPM (Node Package Manager, Node 包管理器) 进行管理。 [p.2]
- **内部模块的组织**: 通过 CommonJS、ES6 Module (ES6 模块规范) 和构建工具 (如 Webpack、Vite) 进行组织。 [p.2]
- **模块源码到目标代码的编译和转换**: 通过 Babel 等转译工具将模块源码编译/转换为浏览器可执行的目标代码。 [p.2]

二、同步与异步编程基础 [pp.7-9]

同步与异步的概念 [p.7]

- **同步** (Synchronous): 代码按照书写顺序依次执行, 前一步完成后才执行下一步。 [p.7]
- **异步** (Asynchronous): 代码执行时不等待上一步操作完成, 而是继续向下执行。当异步操作完成后通过回调等方式通知结果。 [p.7]
- **核心问题**: 异步代码**无法通过 return 返回值**, 因为 return 执行时异步操作尚未完成。 [p.7]

回调函数的由来 [p.8]

- 为解决异步代码无法通过 `return` 返回值的问题，引入了**回调函数** (Callback Function) 的设计模式。 [p.8]
- 回调函数：将一个函数作为参数传入异步操作，当异步操作完成后，调用该函数处理结果。 [p.8]
- 回调函数本质上是将**后续逻辑**封装为函数传入异步操作。 [p.8]

回调地狱 [p.9]

- **回调地狱** (Callback Hell)：多次嵌套回调函数造成的问题。当多个异步操作需要顺序执行时，回调层层嵌套，导致代码层层缩进。 [p.9]
- 主要危害：
 - 代码可读性极差，难以理解执行顺序。
 - 调试困难，错误追踪不便。
 - 维护成本高，添加或修改逻辑容易出错。
- 这是 Promise 出现的重要动因。 [p.9]

三、Promise 对象详解 [pp.6–15]

Promise 简介 [p.6]

- Promise 对象是**异步编程的一种解决方案**，比传统的解决方案（事件处理 + 回调函数）更合理和强大。 [p.6]
- **历史**：由社区最早提出和实现，**ECMAScript** 将其写进了语言标准，统一和规范了用法，原生提供 Promise 对象。 [p.6]
- Promise 对象是一个**用于存取数据的容器**，在异步处理中用于封装传统的基于 XMLHttpRequest 对象进行异步请求后的各种状态和值。 [p.6]
- 从语法上说：Promise 是一个对象，从中可以**获取异步操作的消息**。Promise 提供**统一的 API**，各种异步操作都可以用同样的方法进行处理。 [p.6]

Promise 的基本用法 [p.10]

- **构造函数**：`new Promise((resolve, reject) => {...})`，Promise 的构造方法**必须有一个函数作为参数**。 [p.10]
- 该函数的两个参数 `resolve` 和 `reject` 也是函数，分别在**成功时**和**失败时**调用进行存值。 [p.10]
- 通过 Promise 实例对象的 `.then()` 方法中的两个回调函数分别在成功时**取值**，或在失败时**处理错误**。 [p.10]
- Promise 对象内部维护两个属性： [p.10]
 - **PromiseResult**：保存异步结果的值。
 - **PromiseState**：用于保存状态（三种：`pending`、`fulfilled`、`rejected`）。
- `.then()` 方法会监听 **PromiseState** 的状态，并选择一个回调函数运行。 [p.10]

Promise 的三种状态 [p.11]

- `pending`（进行中）：初始状态，既未成功也未失败。 [p.11]
- `fulfilled`（已成功）：操作成功完成。 [p.11]
- `rejected`（已失败）：操作失败。 [p.11]

Promise 的主要特点 [p.11]

1. **状态不受外界影响**：Promise 对象代表一个异步操作，只有异步操作的结果可以决定当前是哪一种状态，**任何其他操作都无法改变这个状态**。 [p.11]

2. 一旦状态改变，任何时候都可以得到这个结果：Promise 对象的状态改变只有两种可能： [p.11]

- 从 pending 变为 fulfilled。
- 从 pending 变为 rejected。

只要这两种情况发生，**状态就凝固了**，不会再变了，会一直保持这个结果，这时就称为 **resolved**（已定型）。 [p.11]

3. 如果状态已定型，再对 **Promise** 对象添加回调函数，也会立即得到这个结果。这与事件机制不同——事件如果错过触发时机再添加监听器不会再次触发。 [p.11]

使用范例一：基本创建与使用 [p.12]

- 演示 `new Promise()` 创建实例，传入带有 `resolve` 和 `reject` 的函数。 [p.12]
- 演示通过 `.then(onFulfilled, onRejected)` 分别处理成功和失败两种情况。 [p.12]

使用范例二：结合 `setTimeout` 模拟异步 [p.13]

- 演示在 Promise 构造函数内使用 `setTimeout` 模拟异步操作。 [p.13]
- 演示通过 `resolve()` 和 `reject()` 在适当时间返回结果。 [p.13]

使用范例三：链式调用替代回调嵌套 [p.14]

- **链式调用** (Chaining)： `.then()` 方法返回的是一个新的 Promise 对象，因此可以继续调用 `.then()`，形成链式结构。 [p.14]
- 链式调用用**扁平化的代码替代了回调函数的层层嵌套**，解决了回调地狱的问题。 [p.14]
- 每个 `.then()` 中的返回值会作为下一个 `.then()` 的输入值传递。 [p.14]

Promise 的其他 API [p.15]

- `Promise.prototype.catch()`：专门用于捕获 `rejected` 状态，等价于 `.then(null, onRejected)`。推荐使用 `.catch()` 而不是在 `.then()` 中传入第二个参数。 [p.15]
- `Promise.prototype.finally()`：无论 Promise 状态是 `fulfilled` 还是 `rejected`，最终都会执行的回调函数。常用于**清理工作**（如隐藏 `loading` 动画）。 [p.15]
- `Promise.all(iterable)`：接受一个 Promise 数组作为参数。
 - 当**全部** Promise 都 `fulfilled` 时，返回一个包含所有结果的数组。 [p.15]
 - 当**任一** Promise `rejected` 时，整体立即变为 `rejected`，返回第一个失败的原因。 [p.15]
 - 适用于多个并行异步操作需要全部完成的场景。 [p.15]

四、`async` / `await` 关键字 [pp.18–21]

`async` 关键字 [p.18]

- `async` 关键字用于快速创建异步函数，其目的是**简化异步函数中 Promise 对象的写法**，是 Promise 的**语法糖** (Syntactic Sugar)。 [p.18]
- 实际使用中，`async` 用于和 `await` 关键字组合实现**异步向同步的转换**。 [p.18]
- `async` 声明的函数始终返回一个 Promise 对象——如果函数直接 `return` 一个普通值，该值会被自动包装为 Promise。 [p.18]

`await` 关键字 [p.19]

- 通过 `await` 关键字调用异步函数时，`await` 会将**执行流阻塞**，等待异步函数执行结束。 [p.19]
- 同时自动从返回的 Promise 对象中**取值并返回**——不需要手动调用 `.then()`。 [p.19]

- 使用规则: [p.19]
 - `await` **只能**用于 `async` 声明的异步函数中。
 - `await` **只阻塞** `async` 函数内部的代码, 不阻塞函数外部的代码。
- 优点: 用**同步的方式**编写和使用异步代码, 代码可读性大幅提升, 逻辑更清晰。 [p.19]
- 缺点: 异步的 `reject` 回调被自动忽略了, 只能用额外的 `try-catch` 去处理失败回调。 [p.19]

`async/await` 使用范例 [p.20]

- 将嵌套的 `.then()` 链改写为顺序的 `await` 调用, 代码更符合直觉 (看起来像同步代码)。 [p.20]
- 错误处理: 用 `try {...} catch (error) {...}` 包裹 `await` 语句, 替代 `.catch()` 的错误处理方式。 [p.20]

`async/await` 使用注意事项 [p.21]

- 需要准确理解执行顺序: `await` 后面的代码等价于 `.then()` 中的回调函数, **不会立即执行**。 [p.21]
- 注意**并行 vs 串行**问题: 多个不相关的异步操作应该使用 `Promise.all()` 并行执行, 避免不必要的串行等待导致性能下降。 [p.21]
- 循环中使用 `await` 会使循环变成串行, 除非业务逻辑需要顺序执行, 否则应考虑并行方案。 [p.21]

五、Fetch API [pp.23–25]

什么是 Fetch API [p.23]

- `fetch()` 是 `XMLHttpRequest (XHR)` 的**升级版**, 用于在 JavaScript 脚本里面发出异步 HTTP 请求。 [p.23]
- `fetch()` 归属于**ES6 标准**规范, 绝大部分现代浏览器都提供原生支持。 [p.23]
- `fetch()` 基于**Promise 对象**实现, 极大的简化了异步请求的代码。 [p.23]

`fetch()` 发送请求 [p.24]

- 完整语法: `fetch(url, optionObject)` [p.24]
 - `url`: 请求的 URL 地址 (字符串)。
 - `optionObject`: 可选的配置对象, 包含 `method`、`headers`、`body` 等。
- **GET 请求**: 可以省略 `optionObject`, 默认即为 GET。 [p.24]
- **POST 请求**: 在 `optionObject` 中指定 `method: 'POST'`, 并在 `headers` 中设置 `Content-Type`, 在 `body` 中传入请求体数据 (通常为 JSON 字符串)。 [p.24]

`fetch()` 的响应对象 [p.25]

- `fetch()` 返回的是一个名为 **Response** 的对象 (包装在 Promise 中)。 [p.25]
- Response 对象的常用属性和方法:
 - `Response.ok` — 布尔值, 对应 HTTP 状态码, 2xx 为 `true`, 其他为 `false`。 [p.25]
 - `Response.status` — 整型, 真实的 HTTP 状态码 (如 200、404、500)。 [p.25]
 - `response.text()` — 返回服务端的文本字符串 (返回 Promise)。 [p.25]
 - `response.json()` — 返回服务端的 JSON 对象 (返回 Promise)。 [p.25]
 - `Response.clone()` — Response 底层通过 **Stream** 流实现, **只能读取一次**, `clone()` 用于创建副本以便多次读取。 [p.25]

Fetch API 的优点与缺点 [p.31]

- 优点:
 - **原生支持**: 现代浏览器原生支持, 无需引入额外依赖库。 [p.31]

- **Promise 风格**: 基于 Promise, 比传统的 XMLHttpRequest 更简洁。 [p.31]
- **更现代化设计**: 原生支持 async/await, 代码风格更清晰。 [p.31]
- **流式处理** (ReadableStream): 支持响应体的流式处理, 适合处理大文件或渐进式加载。 [p.31]
- **缺点**:
 - **不自动处理 JSON**: 需要手动调用 res.json() 解析数据。 [p.31]
 - **错误处理不友好**: fetch 只在网络错误 (如断网) 时会 reject, 对于 HTTP 错误状态码 (如 404、500) 不会抛错, 需要手动判断 response.ok。 [p.31]
 - **不支持请求取消**: 早期版本不支持取消请求 (现在需要配合 AbortController 实现)。 [p.31]
 - **缺乏对旧浏览器的支持**: 比如 IE 不支持。 [p.31]

六、Axios 库 [pp.28–29]

Axios 简介 [p.28]

- Axios 是一个基于 Promise 的**第三方 HTTP 库**, 可以用在浏览器和 Node.js 中。 [p.28]
- Axios 本质上也是对原生 XHR 的封装, 只不过是**Promise 的实现版本**。 [p.28]
- Axios 封装后提供更好用更便捷的 API: [p.28]
 - **拦截请求和响应** (Interceptors): 允许在请求发送前和响应返回后统一添加处理逻辑 (如添加 token、日志记录、统一的错误处理)。
 - **转换请求数据和响应数据** (Transform): 自动对请求数据和响应数据进行转换处理。
 - **自动转换 JSON 数据**: 请求时自动将 JS 对象序列化为 JSON, 响应时自动将 JSON 反序列化为 JS 对象。

Axios 使用范例要点 [p.29]

- axios.get(url): 发送 GET 请求, 返回 Promise, 可直接 await 获取响应。 [p.29]
- axios.post(url, data): 发送 POST 请求, 第二个参数为请求体数据。 [p.29]
- axios.create({baseUrl, timeout, headers}): 创建带有**默认配置**的 Axios 实例, 便于复用。 [p.29]
- 拦截器使用: 通过 axios.interceptors.request.use() 和 axios.interceptors.response.use() 添加请求/响应拦截器, 实现统一的 token 注入、错误处理等。 [p.29]

Axios 的优点与缺点 [p.32]

- **优点**:
 - **自动转换 JSON**: 请求和响应数据会自动进行 JSON 转换, 开发者无需手动序列化/反序列化。 [p.32]
 - **更友好的错误处理**: HTTP 错误 (如 404、500) 会自动进入 .catch(), 便于统一处理。 [p.32]
 - **支持请求取消**: 通过 CancelToken 或 AbortController 支持取消在途请求。 [p.32]
 - **请求和响应拦截器**: 方便统一添加 token、处理响应、日志记录等。 [p.32]
 - **更丰富的配置选项**: 如设置超时 timeout、自定义请求头、转换函数等。 [p.32]
 - **更好的浏览器兼容性**: 对老版本浏览器 (如 IE) 更友好。 [p.32]
- **缺点**:
 - **需要额外引入库**: 体积虽不是特别大, 但比 fetch (原生) 多了一个依赖。 [p.32]
 - **不是原生的**: 增加项目依赖, 项目越大依赖管理问题越明显。 [p.32]

七、Fetch API 与 Axios 对比 [pp.31–32]

下面从多个维度对比 `fetch()`（原生 API）与 `Axios`（第三方库）的主要差异：

对比维度	Fetch API（原生） [p.31]	Axios（第三方库） [p.32]
来源	ES6 规范，浏览器原生提供，无需安装。	第三方库，需要 <code>npm install axios</code> 引入。
JSON 处理	不自动处理，需手动调用 <code>.json()</code> 解析。	自动转换 ，请求和响应自动序列化/反序列化。
错误处理	不友好 ：仅网络错误 <code>reject</code> ，HTTP 错误码（404、500）不会抛错，需手动判断 <code>response.ok</code> 。	友好 ：HTTP 错误自动进入 <code>.catch()</code> ，便于统一处理。
请求取消	早期不支持，现在需配合 <code>AbortController</code> 。	支持 <code>CancelToken</code> 和 <code>AbortController</code> 两种方式。
拦截器	无。	有 ：请求和响应拦截器，统一添加 <code>token</code> 、日志等。
超时设置	无原生支持，需手动通过 <code>AbortController</code> 实现。	内置 <code>timeout</code> 配置，直接设置超时毫秒数。
请求/响应转换	无内置转换，需手动处理。	内置 <code>transformRequest / transformResponse</code> 。
浏览器兼容性	不支持 IE。	对老版本浏览器更友好（支持 IE）。
体积/依赖	0 依赖 （原生 API）。	需要额外引入库文件，增加打包体积。
流式处理	支持 <code>ReadableStream</code> ，适合大文件和渐进式加载。	提供 <code>responseType: 'stream'</code> 支持。
取消请求	<code>AbortController</code> （较新标准）。	<code>CancelToken</code> （传统）+ <code>AbortController</code> （新）双支持。
使用场景	简单请求、不需要拦截器、追求零依赖。	复杂项目、需要拦截器、统一配置、错误处理。
选择建议	小型项目、原型开发、注重包体积。	中大型项目、需要统一 <code>token</code> 管理、拦截器处理。

八、项目案例与前端项目演进 [pp.4, 34–39]

项目案例：宠物商城首页

- 贯穿全章的实践案例，分别在第 4 页、第 16 页、第 26 页、第 34 页出现，展示前端项目从零到组件化的完整流程。[pp.4, 16, 26, 34]
- 案例将 `Promise`、`Fetch API`、`Axios` 等异步请求技术与实际业务场景结合。[p.16, 26]

第一步：JavaScript 功能的拆解 [pp.35–36]

- 将整体业务逻辑拆分为**独立的功能模块**。 [p.35]
- 每个功能由一个或多个 JavaScript 文件负责实现。 [p.35]
- 需要理清模块之间的**依赖关系**和调用顺序 [p.36]:
 - 哪些模块是独立的，可以并行加载？
 - 哪些模块依赖其他模块，必须按顺序加载？
- 功能拆解使得**代码组织清晰**，便于维护和复用。 [p.36]
- 这是项目从”一个大文件”到”多个有组织文件”的第一步演进。 [p.36]

第二步：静态资源的打包 [p.37]

- 使用构建工具（如**Webpack**、**Vite**、Rollup 等）对静态资源进行**打包** (Bundle)。 [p.37]
- 打包的核心目的：
 - **合并文件**：将多个 JS/CSS 文件合并为一个或少数几个文件。
 - **压缩体积**：去除注释、空格，缩短变量名，减小文件大小。
 - **语法转换**：通过 Babel 等转译工具将 ES6+ 语法转为兼容旧浏览器的 ES5 语法。
 - **优化加载性能**：代码分割 (Code Splitting)、Tree Shaking 等。
- 效果：减少 HTTP 请求次数，提升页面加载速度。 [p.37]

第三步：模块化和组件化 [p.38]

- **模块化** (Modularization):
 - 按**功能**划分代码，一个模块对应一个或多个 JS 文件。
 - 关注**代码组织和复用**，通过 `import/export` 管理依赖。
- **组件化** (Componentization):
 - 按**视觉/交互单元**划分，一个组件包含 HTML + CSS + JS。
 - 关注**UI 封装和复用**，组件之间通过 `props/events` 进行通信。
- 两者的区别与联系：
 - 模块化侧重于代码层面的逻辑拆分。
 - 组件化侧重于 UI 层面的视觉单元拆分。
 - 组件化通常建立在模块化的基础之上。
- 两者结合使项目**结构清晰**、**可维护性高**、**利于团队协作**。 [p.38]

未来演进方向 [p.39]

架构方面 [p.39]

- **MPA vs SPA**:
 - **MPA** (Multi-Page Application, 多页应用)：每个页面独立加载，SEO 友好，但页面切换有白屏。
 - **SPA** (Single-Page Application, 单页应用)：用户体验流畅（无刷新切换页面），但首屏加载慢、SEO 不友好。
- **SSR** (Server-Side Rendering, 服务端渲染):
 - SPA 首屏效率不高的问题催生了 SSR 的出现。
 - SSR 在服务端完成首屏渲染，兼顾了**首屏速度**和**SEO**。
 - 主流方案：Next.js (React)、Nuxt.js (Vue)。

技术方面 [p.39]

- 渲染数据的方法：

- 从 **MVVM** (Model-View-ViewModel) 模式演进到**双向绑定** (Two-way Data Binding)。
- MVVM 将视图 (View) 与数据模型 (Model) 分离, 通过 ViewModel 进行双向通信。
- 双向绑定: 数据变化自动更新视图, 视图变化自动更新数据 (如 Vue 的 `v-model`)。
- **SPA 的路由问题:**
 - **前端路由** (Frontend Routing): 实现在不刷新页面的情况下切换页面内容。
 - 两种实现方式: **Hash 路由** (URL 中 # 后的部分) 和 **History API** (HTML5 的 `pushState/replaceState`)。
- **SPA 中数据的保存与状态保持:**
 - SPA 中跨页面、跨组件的数据管理成为核心挑战。
 - 解决方案: 状态管理库如 **Vuex** (Vue)、**Redux** (React)、**Pinia** (Vue3)。
 - 关注**数据状态的一致性和单向数据流**。

九、本章小结 [p.40]

本章围绕三个核心主题展开:

1. 异步请求相关内容:

- **Promise 对象:** 异步编程的基础, 三种状态 (pending / fulfilled / rejected), 链式调用。[pp.6–15]
- **async/await 语法糖:** 用同步方式编写异步代码, 配合 try-catch 处理错误。[pp.18–21]
- **Fetch API:** 浏览器原生异步请求能力。[pp.23–25]
- **Axios 库:** 第三方功能丰富的 HTTP 客户端。[pp.28–29]

2. Fetch vs Axios 对比分析:

- Fetch 原生支持、零依赖; Axios 功能更强大、错误处理更友好。[pp.31–32]
- 根据项目规模和需求选择合适的技术方案。

3. 前端项目的演进路径:

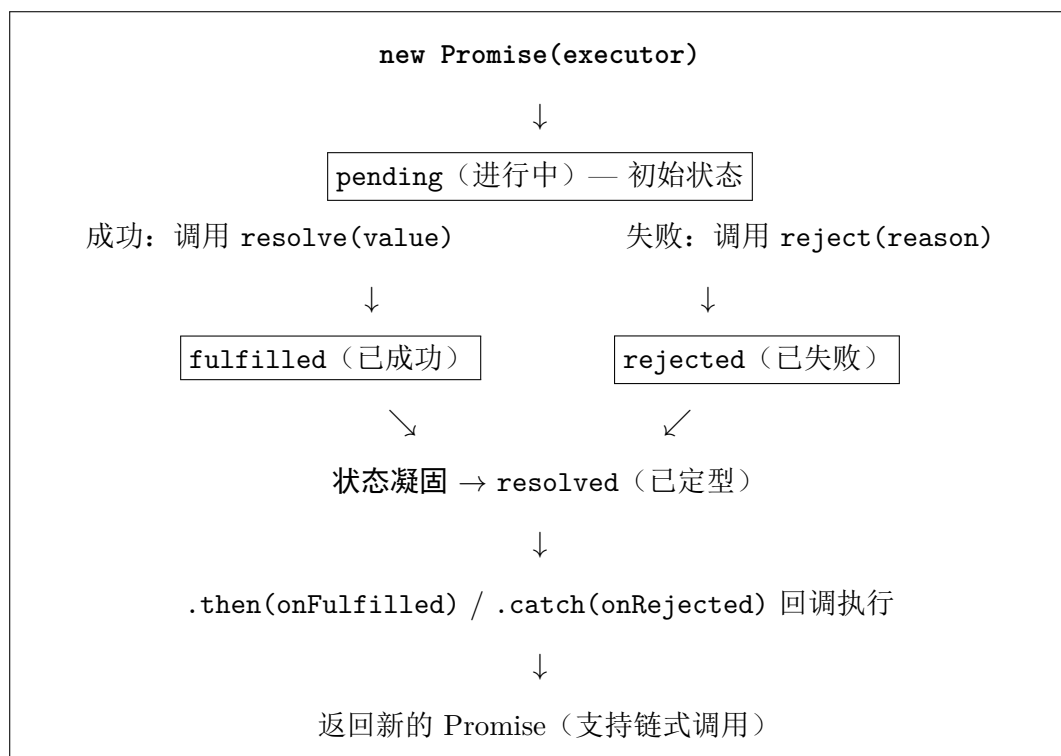
- JavaScript 功能拆解 → 静态资源打包 → 模块化与组件化。[pp.35–38]
- 未来方向: SPA/SSR/MVVM/双向绑定/前端路由/状态管理。[p.39]

附录 A：核心 API 速查表

API / 语法	说明与用法
<code>new Promise((resolve, reject) => {...})</code>	创建 Promise 实例。 <code>resolve</code> 在成功时调用， <code>reject</code> 在失败时调用。[pp.6, 10]
<code>promise.then(onFulfilled, onRejected)</code>	监听 Promise 状态变化，指定成功/失败回调。返回新 Promise 实现链式调用。[p.10]
<code>promise.then(onF).catch(onError)</code>	链式调用 + 错误捕获模式。推荐使用 <code>.catch()</code> 而非在 <code>.then()</code> 中传第二个参数。[p.15]
<code>promise.finally(callback)</code>	无论 Promise 成功或失败，最终都会执行的回调。常用于清理工作。[p.15]
<code>Promise.all([p1, p2, ...])</code>	全部 Promise 成功则返回结果数组；任一失败则整体 rejected。[p.15]
<code>Promise.race([p1, p2, ...])</code>	返回第一个完成的 Promise 的结果（无论成功或失败）。[p.15]
<code>Promise.resolve(value)</code>	返回一个状态为 fulfilled 的 Promise 对象。用于快速将值包装为 Promise。[p.15]
<code>Promise.reject(reason)</code>	返回一个状态为 rejected 的 Promise 对象。[p.15]
<code>async function fn() {...}</code>	声明异步函数，函数始终返回 Promise。是 Promise 的语法糖。[p.18]
<code>await promise</code>	阻塞等待 Promise 结果并自动取值。只能用于 async 函数内部。需 try-catch 处理 reject。[p.19]
<code>fetch(url, {method, headers, body})</code>	原生 HTTP 请求 API (ES6)，返回 <code>Response</code> 对象（包装在 Promise 中）。[p.24]
<code>response.ok</code>	布尔值，HTTP 2xx 为 true，其他为 false。[p.25]
<code>response.status</code>	整型，真实 HTTP 状态码。[p.25]
<code>response.text()</code>	解析响应体为文本字符串（返回 Promise）。[p.25]
<code>response.json()</code>	解析响应体为 JSON 对象（返回 Promise）。[p.25]
<code>response.clone()</code>	克隆 Response 对象（因为 Stream 只能读取一次）。[p.25]
<code>axios.get(url)</code>	Axios GET 请求发送。返回 Promise，自动 JSON 解析。[p.29]
<code>axios.post(url, data, config)</code>	Axios POST 请求发送。 <code>data</code> 自动序列化为 JSON。[p.29]
<code>axios.create({baseUrl, timeout})</code>	创建带默认配置的 Axios 实例，便于在项目中统一复用。[p.29]
<code>axios.interceptors.request.use(fn)</code>	请求拦截器：在请求发送前执行（如添加 token 到请求头）。[p.29]
<code>axios.interceptors.response.use(fn)</code>	响应拦截器：在响应返回后执行（如统一错误处理、刷新 token）。[p.29]

附录 B: Promise 状态转换图

Promise 对象的生命周期中有且仅有三种状态，状态转换不可逆：



关键规则：

1. 状态只能从 `pending` 转换到 `fulfilled` 或 `rejected`，**不能反向转换**。[p.11]
2. 状态一旦改变就**凝固** (settled/resolved)，不会再发生变化。[p.11]
3. 状态定型后添加的回调函数**仍会立即执行**，这是 Promise 与事件机制的重要区别。[p.11]
4. `.then()` 和 `.catch()` 始终**返回一个新的 Promise**，这是链式调用的基础。[p.14]

附录 C: Fetch vs Axios 决策速查

对比维度	Fetch API [p.31]	Axios [p.32]
来源	ES6 原生，浏览器内置	第三方库， <code>npm install axios</code>
JSON 处理	手动调用 <code>.json()</code>	自动转换
错误处理	仅网络错误 <code>reject</code> ，HTTP 错误需手动判断 <code>ok</code>	HTTP 错误自动进入 <code>.catch()</code>
请求取消	<code>AbortController</code> (较新)	<code>CancelToken</code> + <code>AbortController</code>
拦截器	无	有 (请求 + 响应)
超时设置	无原生支持	内置 <code>timeout</code> 配置
兼容性	不支持 IE	更友好 (支持 IE)
体积	0 依赖	需要额外引入
流式处理	支持 <code>ReadableStream</code>	支持 <code>responseType: 'stream'</code>

使用建议:

- 简单需求 / 原型开发: 使用 `fetch()`, 零依赖, 原生支持, 代码简洁。 [p.31]
- 复杂项目 / 统一管理: 使用 `Axios`, 拦截器、自动 JSON、超时设置、更好错误处理。 [p.32]
- 如果需要更好的流式处理: 优先考虑 `fetch()`, 其原生 `ReadableStream` 支持更好。 [p.31]

复习建议

- 熟练掌握 `Promise` 的三种状态及状态转换规则, 理解状态不可逆和定型后可立即取值的特点。 [pp.10–11]
- 能够手写基本的 `Promise` 创建、`.then()` 链式调用、`.catch()` 错误捕获代码。 [pp.10–15]
- 理解 `async/await` 是 `Promise` 的语法糖, 能用 `try-catch` 替代 `.catch()` 的错误处理。 [pp.18–21]
- 掌握 `Fetch API` 的基本用法: 完整语法、GET/POST 请求、`Response` 对象关键属性和方法。 [pp.23–25]
- 了解 `Axios` 的核心特性: 拦截器、自动 JSON 转换、错误处理、请求取消。 [pp.28–29]
- 能从多个维度对比 `Fetch` 和 `Axios` 的优劣, 根据场景做出合理选择。 [pp.31–32]
- 理解前端项目演进的三个阶段: 功能拆解 → 资源打包 → 模块化与组件化。 [pp.35–38]
- 了解未来演进方向中的关键概念: SPA、SSR、MVVM、双向绑定、前端路由、状态管理。 [p.39]
- 建议对照教材幻灯片中的代码范例进行动手练习, 特别是 pp.12–14、p.20–21、p.29 的代码。 [pp.12–29]